

=====

MYSQL SERVER - SPECIFIC VERSION INSTALLATION RUNBOOK

UBUNTU / PRODUCTION DBA PROCEDURE

=====

PURPOSE

This runbook explains how to install a SPECIFIC MySQL version on Ubuntu safely.

Example requirement:

Client: "Install MySQL Server 8.4.7"

The procedure covers:

1. Requirement gathering
2. Ubuntu checks
3. MySQL checks
4. Repository/package verification
5. Exact version verification
6. Installation
7. Package verification
8. MySQL service verification
9. Database verification
10. Configuration checks
11. Production/no-downtime approach
12. Rollback considerations

=====

IMPORTANT DBA RULE

=====

DO NOT run:

```
sudo apt install mysql-server
```

without first checking which version Ubuntu/repository will install.

The client may have requested a specific version.

Always verify the exact approved version first.

=====

1. COLLECT CLIENT REQUIREMENTS

=====

Before installation, get the following information:

MySQL Version:

Ubuntu Version:

Architecture:

Environment:

DEV / UAT / PROD

Hostname:

Server IP:

MySQL Port:

Data Directory:

Character Set:

Collation:

Example:

MySQL Version: 8.4.7

Ubuntu: 24.04

Architecture: amd64

Environment: PROD

IMPORTANT:

Do not install a different version without client/change approval.

=====

2. CHECK UBUNTU VERSION

=====

Run:

```
cat /etc/os-release
```

Also:

```
lsb_release -a
```

Record:

Ubuntu Version:

```
=====
3. CHECK SERVER ARCHITECTURE
=====
```

Run:

```
uname -m
```

And:

```
dpkg --print-architecture
```

Typical result:

```
amd64
```

Record:

```
Architecture:
```

```
=====
4. CHECK WHETHER MYSQL IS ALREADY INSTALLED
=====
```

Run:

```
mysql --version
```

Then:

```
dpkg -l | grep -i mysql
```

Also:

```
dpkg -l | grep -i percona
```

IMPORTANT:

If MySQL/Percona is already installed, determine whether this is:

```
NEW INSTALLATION
```

or:

```
UPGRADE / PATCH
```

Do not treat an existing production server as a new installation.

=====

5. CHECK MYSQL SERVICE

=====

Run:

```
sudo systemctl status mysql --no-pager
```

Check:

```
sudo systemctl is-active mysql
```

If MySQL is already running, record:

Current MySQL Version:

=====

6. CHECK MYSQL VERSION FROM DATABASE

=====

Login:

```
mysql -uroot -p
```

Run:

```
SELECT VERSION();
```

Also:

```
SHOW VARIABLES LIKE 'version%';
```

Record:

Current MySQL Version:

=====

7. DETERMINE WHETHER USING ORACLE MYSQL OR PERCONA

=====

Run:

```
SELECT VERSION();
```

```
SHOW VARIABLES LIKE 'version_comment';
```

Example Oracle MySQL:

MySQL Community Server - GPL

Example Percona:

Percona Server

IMPORTANT:

Use the correct vendor repository/package for the installed database software.

Do not mix Oracle MySQL and Percona Server packages unless the migration/upgrade path has been specifically validated.

```
=====
8. CHECK DISK SPACE
=====
```

Run:

```
df -h
```

Check:

```
df -h /
```

Check inode usage:

```
df -i
```

Make sure sufficient disk space exists before installation.

```
=====
9. CHECK APT PACKAGE INFORMATION
=====
```

Run:

```
apt-cache policy mysql-server
```

Also:

```
apt list -a mysql-server
```

This shows which MySQL versions are available from the configured repositories.

=====

10. VERIFY THE EXACT CLIENT VERSION

=====

Example:

Client requested:

MySQL 8.4.7

Check available versions:

apt list -a mysql-server

You need to find the exact approved version.

Example:

mysql-server/...
8.4.7-1ubuntu24.04
8.4.6-1ubuntu24.04

If 8.4.7 is not available:

STOP.

Do NOT automatically install 8.4.8.

Contact the client/change owner and get approval for the alternate version.

=====

11. CHECK PACKAGE VERSION BEFORE INSTALLATION

=====

Run:

apt-cache policy mysql-server

Record:

Candidate Version:

Requested Version:

Confirm:

Candidate/available version
=
Approved client version

Only continue if the version is approved.

=====
12. SIMULATE INSTALLATION
=====

Before actually installing, use:

```
sudo apt install --simulate mysql-server=<VERSION>
```

Example:

```
sudo apt install --simulate \  
mysql-server=8.4.7-1ubuntu24.04
```

Review the packages that will be:

Installed
Upgraded
Removed

IMPORTANT:

If the simulation shows unexpected package removals,
STOP and investigate.

=====
13. CHECK WHAT WILL CHANGE
=====

Run:

```
apt-cache policy mysql-server
```

Then:

```
apt list --upgradable
```

Review carefully.

Do not blindly run:

```
sudo apt upgrade
```

on a production database server.

=====

14. INSTALL SPECIFIC VERSION

=====

ONLY after confirming the exact approved version.

Example:

```
sudo apt install \  
mysql-server=8.4.7-1ubuntu24.04
```

IMPORTANT:

Replace the version with the EXACT version returned by:

```
apt list -a mysql-server
```

Do not copy the example version blindly.

=====

15. IF DEPENDENCY ERROR OCCURS

=====

If apt reports dependency problems:

```
sudo apt-get -f install
```

Then check:

```
sudo dpkg --configure -a
```

Then:

```
sudo apt-get update
```

IMPORTANT:

Do not continue blindly if package conflicts occur.

Review the error first.

=====

16. VERIFY MYSQL PACKAGE

=====

Run:

```
dpkg -l | grep -i mysql
```


You may see packages such as:

```
mysql-server
mysql-client
mysql-server-core
mysql-client-core
```

Check the installed package version:

```
dpkg -s mysql-server
```

Look for:

Version:

```
=====
17. VERIFY MYSQL VERSION
=====
```

Run:

```
mysql --version
```

Then:

```
mysql -uroot -p
```

Run:

```
SELECT VERSION();
```

The result MUST match the approved version.

```
=====
18. CHECK MYSQL SERVICE
=====
```

Run:

```
sudo systemctl status mysql --no-pager
```

Then:

```
sudo systemctl is-active mysql
```

Expected:

active

If inactive:

STOP

Check:

```
sudo journalctl -u mysql -n 100 --no-pager
```

Also check:

```
sudo tail -100 /var/log/mysql/error.log
```

```
=====
19. CHECK MYSQL PORT
=====
```

Run:

```
sudo ss -lntp | grep 3306
```

Expected MySQL listening on the configured port.

Check from MySQL:

```
mysql -uroot -p
```

```
SHOW VARIABLES LIKE 'port';
```

```
=====
20. CHECK MYSQL DATA DIRECTORY
=====
```

Run:

```
mysql -uroot -p
```

Then:

```
SHOW VARIABLES LIKE 'datadir';
```

Record:

Datadir:

=====

21. CHECK BIND ADDRESS

=====

Run:

```
mysql -uroot -p
```

Then:

```
SHOW VARIABLES LIKE 'bind_address';
```

Make sure the configuration is appropriate for your environment.

=====

22. CHECK CHARACTER SET

=====

Run:

```
mysql -uroot -p
```

Then:

```
SHOW VARIABLES LIKE 'character_set_server';
```

Record:

Character Set:

=====

23. CHECK COLLATION

=====

Run:

```
SHOW VARIABLES LIKE 'collation_server';
```

Record:

Collation:

=====

24. CHECK MYSQL ERROR LOG

=====

Run:

```
sudo journalctl -u mysql --since "30 minutes ago"
```

And:

```
sudo tail -100 /var/log/mysql/error.log
```

Look for:

- ERROR
- Warning
- InnoDB errors
- Permission errors
- Disk errors
- Configuration errors

```
=====
25. CHECK MYSQL DATABASES
=====
```

Login:

```
mysql -uroot -p
```

Run:

```
SHOW DATABASES;
```

Check that expected system databases exist.

For example:

- information_schema
- mysql
- performance_schema
- sys

```
=====
26. BASIC MYSQL HEALTH CHECK
=====
```

Run:

```
mysqladmin -uroot -p ping
```

Expected:

```
mysql is alive
```

Then:

```
mysql -uroot -p
```

Run:

```
SHOW GLOBAL STATUS LIKE 'Threads_connected';
```

```
SHOW GLOBAL STATUS LIKE 'Threads_running';
```

```
SHOW GLOBAL STATUS LIKE 'Uptime';
```

```
=====
27. CHECK MYSQL CONFIGURATION FILES
=====
```

Common location:

```
/etc/mysql/
```

Check:

```
sudo ls -lah /etc/mysql/
```

Common configuration file:

```
/etc/mysql/mysql.conf.d/mysqld.cnf
```

Review:

```
sudo cat /etc/mysql/mysql.conf.d/mysqld.cnf
```

DO NOT overwrite an existing production configuration without first taking a copy.

```
=====
28. BACKUP CONFIGURATION BEFORE CHANGING IT
=====
```

If configuration changes are required:

```
sudo cp \
/etc/mysql/mysql.conf.d/mysqld.cnf \
/etc/mysql/mysql.conf.d/mysqld.cnf.backup_$(date +%Y%m%d_%H%M%S)
```

Verify:

```
sudo ls -lah /etc/mysql/mysql.conf.d/
```

=====

29. CHECK MYSQL PACKAGE HOLD

=====

Check:

```
apt-mark showhold
```

If MySQL packages are held:

```
mysql-server  
mysql-client
```

Do not remove the hold without understanding why it was created.

=====

30. HOLD SPECIFIC VERSION IF REQUIRED

=====

If the client requires the server to remain at a specific version:

```
sudo apt-mark hold mysql-server mysql-client
```

Check:

```
apt-mark showhold
```

IMPORTANT:

Coordinate this with your organization's patch management process.

A package hold can prevent future security updates.

=====

31. UNHOLD WHEN PATCHING IS APPROVED

=====

When a future approved patch is required:

```
sudo apt-mark unhold mysql-server mysql-client
```

Then verify:

```
apt-mark showhold
```

=====

32. PRODUCTION SERVER - DO NOT PATCH DIRECTLY

=====

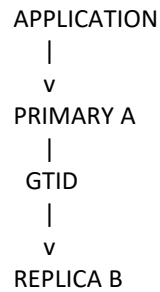
If the server is an existing production PRIMARY and the client requires minimum/no downtime:

DO NOT simply run:

`sudo apt upgrade`

Instead use a rolling patch strategy.

Architecture:



Patch B first.

33. PRODUCTION ROLLING PATCH PROCEDURE

STEP 1:

Verify replication is healthy.

On Replica B:

```
SHOW REPLICA STATUS\G
```

Expected:

```
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
```

```
Last_IO_Error:
Last_SQL_Error:
```

STEP 2:

Take XtraBackup of Replica B.

Example:

```
sudo xtrabackup \
  --backup \
  --target-dir=/backup/mysql/prepatch_$(date +%Y%m%d_%H%M%S)
```

STEP 3:

Patch Replica B.

STEP 4:

Verify MySQL version on B.

mysql --version

STEP 5:

Verify replication after patch.

SHOW REPLICA STATUS\G

STEP 6:

Stop application writes.

STEP 7:

Confirm Replica B has completely caught up.

STEP 8:

Stop replication on B.

STOP REPLICA;

STEP 9:

Promote B.

SET GLOBAL read_only=OFF;

SET GLOBAL super_read_only=OFF;

STEP 10:

Redirect application traffic to B.

STEP 11:

Test application.

STEP 12:

Rebuild/reconfigure old Primary A as Replica of B.

STEP 13:

Verify B -> A replication.

STEP 14:

Patch A.

STEP 15:

Verify A replication.

STEP 16:

If required, fail back from B -> A.

=====
34. PRODUCTION ROLLING PATCH ARCHITECTURE
=====

START:

A = PRIMARY
B = REPLICA

A
|
| GTID
v
B

PATCH B:

A
|
| GTID
v
B
PATCHED

FAILOVER:

B = PRIMARY
A = OLD PRIMARY

REBUILD:

B = PRIMARY
|
| GTID
v
A = REPLICA

PATCH A:

B = PRIMARY
|
| GTID
v
A = PATCHED REPLICA

FAILBACK:

A = PRIMARY
|
| GTID
v
B = REPLICA

=====

35. IMPORTANT VERSION COMPATIBILITY CHECK

=====

Before installing a specific MySQL version, verify:

Ubuntu version
MySQL version
Architecture
Repository
Package dependencies
Supported upgrade path

Do not assume:

Ubuntu supports every MySQL release.

=====

36. DO NOT MIX PACKAGES

=====

Do not randomly mix:

Oracle MySQL
Percona Server
MariaDB

Example:

DO NOT install a Percona package over an Oracle MySQL installation
unless you are following a documented and tested migration path.

=====

37. DPKG COMMANDS

=====

Check dpkg:

`dpkg --version`

Check architecture:

`dpkg --print-architecture`

Check installed MySQL packages:

`dpkg -l | grep -i mysql`

Check installed Percona packages:

`dpkg -l | grep -i percona`

Get package details:

`dpkg -s mysql-server`

Install a local .deb:

`sudo dpkg -i package.deb`

If dependencies need fixing:

`sudo apt-get -f install`

Configure pending packages:

`sudo dpkg --configure -a`

IMPORTANT:

For normal Ubuntu installation, prefer APT/vendor repositories over manually installing individual .deb files.

=====

38. APT COMMANDS

=====

Update repository information:

`sudo apt update`

Check available MySQL versions:

`apt list -a mysql-server`

Check package policy:

```
apt-cache policy mysql-server
```

Simulate installation:

```
sudo apt install --simulate mysql-server=<VERSION>
```

Install exact version:

```
sudo apt install mysql-server=<VERSION>
```

List upgradeable packages:

```
apt list --upgradable
```

```
=====
39. POST-INSTALLATION CHECKLIST
=====
```

- ☐ Correct Ubuntu version
- ☐ Correct architecture
- ☐ Correct MySQL vendor
- ☐ Correct MySQL version
- ☐ Correct package version
- ☐ MySQL service active
- ☐ MySQL port listening
- ☐ Data directory correct
- ☐ Character set correct
- ☐ Collation correct
- ☐ Configuration reviewed
- ☐ Error log reviewed
- ☐ Database login successful
- ☐ SHOW DATABASES successful
- ☐ mysqladmin ping successful
- ☐ Application connection tested
- ☐ Monitoring configured
- ☐ Backup configured
- ☐ Change ticket updated

=====

40. ROLLBACK PLAN

=====

Before production installation/upgrade, document:

Current MySQL version:

Target MySQL version:

Backup location:

XtraBackup completed:

YES / NO

Backup verified:

YES / NO

Configuration backup:

Rollback owner:

Maintenance window:

If upgrade fails:

1. Do not repeatedly restart MySQL.
2. Check MySQL error log.
3. Check package status.
4. Check disk space.
5. Check configuration.
6. Follow approved rollback procedure.
7. Restore from backup if required.

=====

41. CHANGE MANAGEMENT INFORMATION

=====

CHANGE NUMBER:

CLIENT:

SERVER:

ENVIRONMENT:

CURRENT VERSION:

TARGET VERSION:

UBUNTU VERSION:

MAINTENANCE START:

MAINTENANCE END:

BACKUP LOCATION:

BACKUP VERIFIED:

YES / NO

PATCH INSTALLED:

YES / NO

MYSQL SERVICE:

ACTIVE / FAILED

POST-CHECK:

PASS / FAIL

ROLLBACK REQUIRED:

YES / NO

PERFORMED BY:

APPROVED BY:

=====
42. FINAL DBA PROCEDURE
=====

WHEN CLIENT SAYS:

"Install MySQL Server version X"

DO THIS:

1. Get exact version.
2. Check Ubuntu version.

3. Check architecture.
4. Check Oracle MySQL vs Percona.
5. Check whether MySQL is already installed.
6. Check current version.
7. Check official/vendor repository.
8. Check available package versions.
9. Confirm exact target package version.
10. Run apt simulation.
11. Take backup if existing server.
12. Install exact approved version.
13. Verify dpkg packages.
14. Verify mysql --version.
15. Verify SELECT VERSION().
16. Verify MySQL service.
17. Verify port.
18. Check error log.
19. Check databases.
20. Test application connection.
21. Update change ticket.
22. Enable monitoring.
23. Confirm backup/recovery procedure.

=====

43. MOST IMPORTANT COMMANDS

=====

Check OS:

```
cat /etc/os-release
```

Check architecture:

```
dpkg --print-architecture
```

Check MySQL:

```
mysql --version
```

Check installed packages:

```
dpkg -l | grep -E 'mysql|percona'
```

Check available versions:

```
apt list -a mysql-server
```

Check candidate:

```
apt-cache policy mysql-server
```

Simulate:

```
sudo apt install --simulate mysql-server=<VERSION>
```

Install:

```
sudo apt install mysql-server=<VERSION>
```

Verify:

```
mysql --version
```

Database verification:

```
mysql -uroot -p
```

```
SELECT VERSION();
```

Service:

```
sudo systemctl status mysql --no-pager
```

Health:

```
mysqladmin -uroot -p ping
```

Port:

```
sudo ss -lntp | grep 3306
```

Logs:

```
sudo journalctl -u mysql -n 100 --no-pager
```

```
=====
44. GOLDEN RULE
=====
```

NEVER assume the latest version is the version the client wants.

ALWAYS:

```
Current Version
|
v
Required Version
|
v
```


Check Compatibility

|

v

Check Repository

|

v

Verify Exact Package

|

v

Backup

|

v

Simulate

|

v

Install

|

v

Verify

|

v

Test

|

v

Close Change

=====
END OF MYSQL SPECIFIC VERSION INSTALLATION RUNBOOK
=====